

EE5332 Mini Project – Project Report

Title: FPGA-Based 2D Convolution using HLS

Team Members:

- Aashim Sikka (EE25M092)
- Abhishek Kerketta (EE25M093)

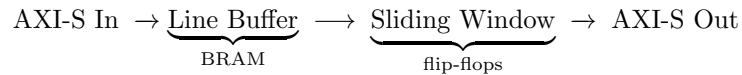
Project Files (Drive Link)

1 Introduction

This report presents the implementation of a 2D convolution accelerator on an FPGA using High-Level Synthesis (HLS). The design streams pixel data through an AXI-Stream interface using DMA and receives kernel coefficients via AXI-Lite. Multiple design variants are explored to analyze the trade-offs between resource utilization and throughput.

2 Design Overview

The baseline design implements a streaming 2D convolution using three nested loops in floating-point precision. It is subsequently converted to fixed-point representation, and the resulting trade-offs in resource utilization and performance are analyzed. For the optimized design, a line buffer and sliding window technique is employed to improve data reuse and throughput. Input pixels are streamed via AXI-Stream using DMA, while kernel coefficients are configured through an AXI-Lite interface. High-Level Synthesis (HLS) pragmas such as loop pipelining, loop unrolling, and array partitioning are applied to further enhance performance.



2.1 Target Platform

- Device: Zynq-7000 (xc7z020clg400-1)
- Tools used: Vitis HLS, Vivado
- Max Image Size: 128 x 128
- Max Kernel Size: 5 X 5

3 Design Variants

We implemented the following variants:

- **Variant 1 (Baseline):** No optimizations
- **Variant 2:** Loop pipelining and unrolling
- **Variant 3:** No pipeline and unrolling

4 Synthesis Results

Table 1: Comparison of Design Variants Across Different Configurations

Variant	Config (Img, Kernel)	LUT	FF	DSP	BRAM	Latency (cycles)	II
Baseline	64x64, 3x3	5356	3694	15	18	10015	10015
Baseline	64x64, 5x5	11051	7090	31	26	20030	20030
Baseline	128x128, 3x3	5435	3706	15	72	10015	10015
Baseline	128x128, 5x5	11212	7115	31	104	20030	20030
Pipelined	64x64, 3x3	5071	2433	15	2	10084	1
Pipelined	64x64, 5x5	11819	17745	31	4	10101	1
Pipelined	128x128, 3x3	5089	2432	15	2	10148	1
Pipelined	128x128, 5x5	11841	17765	31	4	10165	1
No Pipeline	64x64, 3x3	4481	1450	12	2	150338	1502
No pipeline	64x64, 5x5	10474	3621	28	4	1020482	10202
No Pipeline	128x128, 3x3	4486	1456	12	2	150466	1502
No Pipeline	128x128, 5x5	10483	3663	28	4	1020738	10202

5 Pareto Trade-off

The trade-off between performance and resource usage is summarized as:

- Baseline: High resource, poor latency
- Pipelined: Balanced design
- Unrolled: Low performance

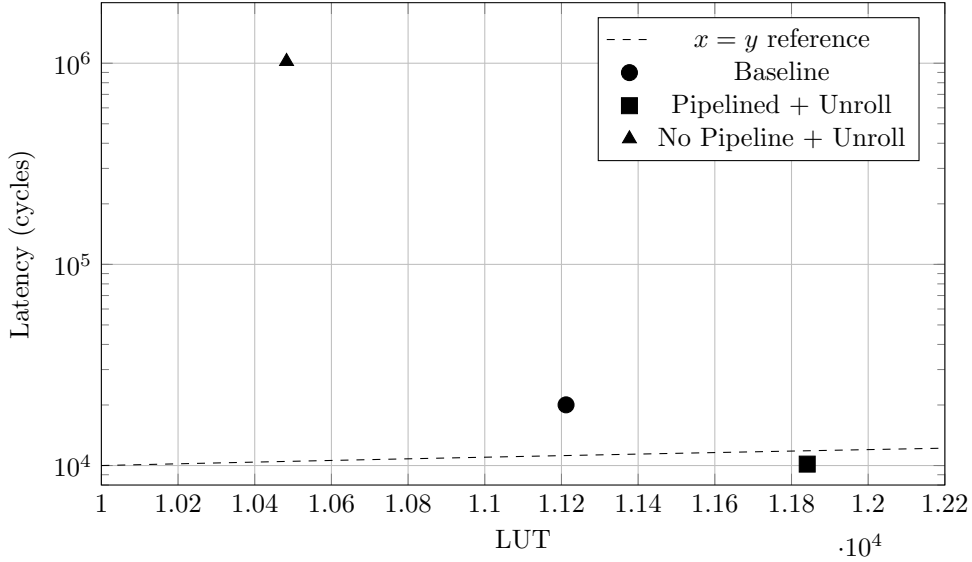


Figure 1: Resource vs Performance Trade-off

6 Hardware Validation

The design was deployed on the PYNQ-Z2 board and tested using a Python/Jupyter notebook. A Python driver was developed to:

1. Load the overlay (`design_1.wrapper.bit + design_1.hwh`) using the PYNQ API.

2. Write kernel coefficients (converted to Q8.8 fixed-point) and image dimensions into the HLS CTRL registers via AXI-Lite.
3. Allocate contiguous DMA buffers and stream the pixel array through the AXI DMA engine.
4. Compare hardware output pixel-by-pixel against a `scipy.ndimage.convolve` software reference (also quantized to Q8.8 precision).

7 Conclusion

128x128 time ~ 64x64 time (expected ~ 4x for O(N²))

Ksz	Kernel	FPGA×	NumPy×	OpenCV×	SciPy×
3×3	Blur 3×3	3.89×	2.80×	2.69×	2.91×
	Gaussian 3×3	3.89×	2.87×	2.40×	2.85×
	Sharpen 3×3	3.89×	2.71×	2.24×	2.70×
	Laplacian 3×3	3.89×	2.99×	2.42×	2.93×
	Sobel X 3×3	3.89×	2.81×	2.40×	2.79×
	Sobel Y 3×3	3.89×	2.66×	2.38×	2.79×
	Emboss 3×3	3.89×	2.89×	2.35×	2.82×
5×5	Blur 5×5	3.81×	3.13×	2.85×	3.17×
	Gaussian 5×5	3.81×	3.21×	2.87×	3.24×
	Sharpen 5×5	3.81×	3.01×	2.61×	2.95×
	Laplacian 5×5	3.81×	2.83×	2.59×	3.01×
	Sobel X 5×5	3.81×	3.12×	2.86×	3.14×
	Sobel Y 5×5	3.81×	3.09×	2.68×	3.13×

Figure 2: Comparison

Method	Avg Time (μs)	× vs FPGA	Verdict
FPGA (5×5)	1223.17	1.0×	FPGA baseline
NumPy	8579.19	7.1×	slower
OpenCV	3369.72	2.8×	slower
SciPy	8700.56	7.2×	slower

Figure 3: Image(128x128), Kernel(5x5)

8 Work Contribution

1. **AASHIM SIKKA:** Baseline C implementation (float, no pragmas); conversion to fixed-point (ap.fixed); implementation of line buffer and sliding window; AXI Stream using DMA; Pynq Deployment
2. **ABHISHEK KERKETTA:** Addition of HLS pragmas and exploration of at least three design points on the Pareto curve; IP export, Vivado TCL scripting, and support for PYNQ deployment; AXI-stream wrapping.